

FedVibroSense

STM32F405 · MPU-6050 · Federated Learning

Complete Operational Field Guide — From Hardware Setup to Data Collection for Publication

What This Guide Covers

This guide walks you through every operational step of the FedVibroSense system: toolchain installation, hardware wiring, CubeMX project generation, building and flashing firmware, sensor calibration, live data collection, labelling vibration classes, running FL training rounds, monitoring the system over UART, operating the Raspberry Pi aggregation server, and exporting a clean dataset for IEEE publication.

STEP 1: Install Toolchain & Environment

Before touching hardware, prepare your Linux development machine (Ubuntu 22.04 or 24.04 recommended). All tools are available from standard package repositories.

1.1 ARM Cross-Compiler

```
sudo apt update && sudo apt install -y gcc-arm-none-eabi binutils-arm-none-eabi
arm-none-eabi-gcc --version # Verify: should print 12.x or 13.x
```

1.2 Flash & Debug Tools

```
sudo apt install -y openocd stlink-tools
sudo cp /usr/share/openocd/contrib/60-openocd.rules /etc/udev/rules.d/
sudo udevadm control --reload-rules && sudo udevadm trigger
```

NOTE: Plug in your ST-Link and run `lsusb | grep -i stm` to confirm detection before proceeding.

1.3 Serial Monitor

```
sudo apt install -y minicom screen
sudo usermod -aG dialout $USER # Allow non-root UART access (re-login required)
```

1.4 Raspberry Pi (Aggregation Server)

```
sudo apt install -y python3-pip && pip3 install pyserial numpy
```

Enable the Raspberry Pi hardware UART: run `sudo raspi-config` → Interface Options → Serial Port → disable login shell, enable hardware port. Then reboot.

STEP 2 Hardware Wiring

Use the wiring table below. All connections are 3.3 V logic. The 4.7 kΩ pull-up resistors on SDA and SCL are mandatory — without them, I2C at 400 kHz is unreliable.

STM32 Pin	MPU-6050	Function	Notes
PB8	SCL	I2C1 Clock	4.7 kΩ pull-up to 3.3 V
PB9	SDA	I2C1 Data	4.7 kΩ pull-up to 3.3 V
PA0	INT	Data Ready	Active low, STM32 pull-up
3.3 V	VCC	Power	100 nF decoupling cap near VCC
GND	GND	Ground	Common ground
GND	AD0	I2C Address	Ties address to 0x68
PA9	—	FL UART TX	→ RPi RXD (GPIO15, Pin 10)
PA10	—	FL UART RX	← RPi TXD (GPIO14, Pin 8)

PA2	—	Debug TX	→ USB-UART adapter RX
PA3	—	Debug RX	← USB-UART adapter TX

NOTE: Raspberry Pi GPIO is 3.3 V — direct connection to PA9/PA10 is safe. For a 5 V USB-UART adapter on the debug port, use a level shifter or check that your adapter has a 3.3 V mode.

STEP 3: Generate CubeMX Base Project

STM32CubeMX generates the HAL driver library and startup code. The custom firmware files from this repo are then copied into it. Follow these settings exactly — any deviation will require changes to main.c.

3.1 Create New Project

1. Open STM32CubeMX → New Project → Select Part: STM32F405RGT6
2. Project Name: FedVibroSense | Toolchain: Makefile | Location: this repo folder
3. Firmware Package: STM32Cube FW_F4 v1.27 or later

3.2 Key Peripheral Settings (refer to CubeMX_Config_Guide.txt for full detail)

Peripheral	Setting	Value
Clock (RCC)	SYSCLK via HSE PLL	168 MHz APB1=42 MHz APB2=84 MHz
FPU	Enabled, Hard ABI	-mfloat-abi=hard -mfpv4-sp-d16
I2C1 (MPU-6050)	Fast Mode 400 kHz	PB8=SCL PB9=SDA No DMA
USART2 (Debug)	115200 baud 8N1	PA2=TX PA3=RX Global IRQ
USART1 (FL comms)	921600 baud 8N1	PA9=TX PA10=RX Global IRQ
TIM2 (µs timer)	Free-run 32-bit	Prescaler=83 Period=0xFFFFFFFF
TIM3 (100 Hz)	Period interrupt	Prescaler=839 Period=999 IRQ prio=1
PA0 (EXTI0)	Falling edge IRQ	MPU-6050 INT Pull-Up prio=2
PD12–PD15	GPIO Output PP	LED status indicators

3.3 Generate & Integrate Custom Files

4. In CubeMX: Project Manager → Code Generator → check 'Copy only necessary library files' and 'Keep User Code'.
5. Click GENERATE CODE. CubeMX creates Core/Src/main.c, stm32f4xx_hal_msp.c, stm32f4xx_it.c, and the full Drivers/ tree.
6. DELETE the generated Core/Src/main.c — replace it entirely with the FedVibroSense main.c.
7. Copy all 8 custom .c files into Core/Src/ and all 8 custom .h files into Core/Inc/.
8. DO NOT modify stm32f4xx_hal_msp.c or stm32f4xx_it.c — CubeMX manages these.

STEP 4 Build Firmware & Flash to STM32

4.1 Build from Terminal

```
cd FedVibroSense_STM32
make all -j4
```

A successful build prints Flash and RAM usage. Expected: Flash ~55 KB (5% of 1 MB), RAM ~98 KB (76% of 128 KB). If you see 'region RAM overflowed', add the CCM attribute to NN_Handle_t in main.c — see README Section 'Out of RAM'.

4.2 Flash via ST-Link

```
make flash      # Uses OpenOCD — recommended
```

```
# OR
make flash_stl # Uses st-flash binary
```

NOTE: If OpenOCD reports 'Error: open failed', check that the udev rules from Step 1.2 were applied and the ST-Link USB was replugged after applying them.

4.3 Verify Boot on Debug UART

```
screen /dev/ttyUSB0 115200
```

On successful boot, you will see the calibration prompt. Keep the sensor board perfectly stationary on a flat surface during the ~2-second calibration window that follows automatically.

Expected output: MPU6050 WHO_AM_I OK → Gyro bias: X=0.00xx ... → App_Init complete

STEP 5 Data Collection for Publication

This step is the core of your research workflow. You will label vibration conditions in real time using single-key UART commands while the STM32 accumulates feature windows.

5.1 Physical Testbed Setup

9. Mount the MPU-6050 rigidly on the motor casing or machine frame using adhesive or a bracket — no foam padding, no loose wires touching the board.
10. Verify the mounting is repeatable across sessions by marking the exact position with a pen.
11. For a 3-class dataset: run the machine in NORMAL state first, then add calibrated imbalance masses to the rotating shaft, then introduce controlled looseness at a fastener joint.
12. For each class, collect at least 200 feature windows (200 seconds of data) for sufficient training samples per IEEE Sensors standards.

5.2 Labeling via UART Command Interface

Open a terminal on USART2 (115200 baud). Before starting each condition, type the corresponding key:

Key	Class	When to Press
0	NORMAL	Machine running at rated speed, no fault — baseline
1	IMBALANCE	After attaching the calibrated mass offset to the rotating shaft
2	LOOSENESS	After intentionally loosening a fastener by 1–2 turns
u	Trigger Upload	Force immediate FL weight upload to server (any time)
s	Status	Print FL round count, loss, server connection status
w	Weight Stats	Print NN min/max/mean weights for sanity check

NOTE: Each feature window is exactly 1 second long. The system buffers 10 labeled windows (FL_LOCAL_EPOCHS) before automatically entering training mode. You do not need to press anything to trigger training — it happens automatically.

5.3 Recommended Collection Protocol

13. Session 1 — Normal class: press '0', run the machine for 5 minutes. The system completes ~30 FL rounds automatically (300 windows / 10 per round).
14. Session 2 — Imbalance class: attach mass, press '1', run for 5 minutes.
15. Session 3 — Looseness class: loosen fastener, press '2', run for 5 minutes.
16. For cross-validation data: repeat all three sessions at a different motor speed or load.
17. Log all UART output to a file for paper evidence:

```
screen -L -Logfile session1_normal.log /dev/ttyUSB0 115200
```

STEP 6 Run the Federated Learning Aggregation Server

The aggregation server runs on a Raspberry Pi connected to the STM32 via UART1 (921600 baud). It receives local weights, computes FedAvg, and broadcasts the updated global model back. The full Python server is included in the README — save it as `fed_server.py` on the RPi.

6.1 Start the Server

```
# On Raspberry Pi — single node research mode
python3 fed_server.py --port /dev/ttyAMA0 --clients 1 --rounds 100
```

```
# Multi-node mode (3 STM32 boards):
python3 fed_server.py --port /dev/ttyAMA0 --clients 3 --rounds 100
```

NOTE: The server MUST be started before the STM32 completes its first FL training round, otherwise the STM32 upload will timeout and the FL client will reset. For single-node research, 'clients 1' is sufficient — FedAvg with one client is equivalent to standard SGD with weight transmission.

6.2 What the Server Prints (Expected Output)

```
Server listening on /dev/ttyAMA0 @ 921600 baud
Expecting 1 client(s), 100 FL rounds
```

```
=== FL Round 1/100 ===
  Waiting for client 1/1...
  Client 1 received — ACK sent
  FedAvg done. W1 mean=-0.001234
  Global model sent to client 1
=== FL Round 2/100 === ...
```

6.3 Save Global Models for Publication

```
# In fed_server.py, add after fedavg() call:
import json
np.save(f'global_model_round_{rnd}.npy', global_weights)
```

This saves each aggregated model as a NumPy array. You can later load these and plot weight convergence across rounds, which is a standard figure in FL publications (Fig. 2 in most IEEE IoT Journal papers).

STEP 7 Real-Time Monitoring & Debug

The UART2 debug interface (115200 baud) prints structured log lines at 1-second intervals. Each line is prefixed with a severity level.

7.1 Log Prefix Reference

- **[INF]** — Normal operational event — window built, inference result, FL round progress
- **[ERR]** — Hardware or protocol error — I2C failure, CRC mismatch, UART timeout
- **[VRB]** — Verbose detail — individual IMU samples, weight values. Disabled by default (set `DEBUG_LEVEL 3` in `Config.h` to enable)

7.2 Key Log Lines to Watch

```
[INF] Window #45 | Pred=1 [N=0.08 I=0.87 L=0.05] | feat=1823µs infer=48µs
```

This line appears every second. 'Pred=1' means class IMBALANCE. The three probabilities should peak clearly on one class — if all three are near 0.33, the network has not yet converged.

```
[INF] FL: train step 8/10 label=1 loss=0.2841
```

Training loss should decrease monotonically within a round. If loss increases across rounds (not just within), reduce the learning rate in `Config.h` from 0.01 to 0.005.

```
[INF] FL: === ROUND 12 COMPLETE ===
```

```
[INF] FL Status: state=IDLE round=12 samples=120 loss=0.1923 server=1
```

'server=1' confirms the Raspberry Pi is reachable. 'server=0' means the ACK was never received — check UART wiring and that `fed_server.py` is running.

7.3 LED Status Indicators (STM32F405 Discovery / Nucleo)

LED / Pin	Color	Meaning
PD12 (Green)	Green	Predicted class: NORMAL
PD13 (Orange)	Orange	Predicted class: IMBALANCE
PD14 (Red)	Red	Predicted class: LOOSENESS
PD15 (Blue)	Blue	FL training/upload/download in progress

STEP 8 Dataset Export & Publication Preparation

8.1 Parse UART Logs into a Clean CSV Dataset

The screen -L log files contain every inference result with class probabilities. Use this Python script on your PC to parse them into a publication-ready CSV:

```
import re, csv
pattern = r'Window #(\d+) \| Pred=(\d) \| N=([\d.]+) I=([\d.]+) L=([\d.]+)'
with open('session1_normal.log') as f, open('dataset.csv','w') as out:
    w = csv.writer(out)
    w.writerow(['window','pred','p_normal','p_imbalance','p_looseness','true_label'])
    for line in f:
        m = re.search(pattern, line)
        if m: w.writerow([*m.groups(), 0]) # true_label=0 for normal
```

8.2 Minimum Dataset for Publication

- **Per class** — 200+ labelled windows (200 seconds continuous machine operation)
- **Total** — 600+ windows across 3 classes before FL training is reported
- **FL rounds** — Report convergence curves for at least 30 rounds per paper figure
- **Repetitions** — Run 3 independent sessions per class for error bar calculation
- **Metrics** — Per-round: training loss, test accuracy, $\|W_{\text{global}} - W_{\text{local}}\|_2$ (weight divergence)

8.3 Figures Standard for IEEE Sensors / IoT Journal

18. Training loss vs. FL round number — one curve per class, shows convergence
19. Confusion matrix — 3×3, computed on the held-out 20% of the dataset
20. Inference latency bar chart — forward pass μ s at different optimisation levels
21. RAM/Flash usage table — compare vs. TFLite Micro baseline
22. Weight divergence $\|W_k - W_{\text{global}}\|_2$ per round — demonstrates FL stability

8.4 Reproducibility Checklist for Paper Submission

- **Seed** — Fixed: Utils_SeedRNG(0xABCD1234) — report this value in the paper
- **Config** — Include Config.h values table in supplementary material
- **Git** — Tag the exact firmware commit used for each experimental run
- **Hardware** — State MPU-6050 DLPF setting, full-scale range, and mounting orientation
- **Server** — State Python + NumPy versions used on Raspberry Pi
- **Dataset** — Upload raw .log files and parsed .csv to a Zenodo or IEEE DataPort repository

End of FedVibroSense Operational Field Guide | For questions, refer to README.md or open a GitHub issue.